Špecifikácia privátneho rezervačného systému nehnuteľností

Bc. Lukáš Bujňák

Obsah

1 Základný popis aplikácie **3**
1.1 Motivácia . . . 3
1.2 Ciele našej aplikácie . . . 3
1.3 Cieľová skupina . . . 3
1.4 Desktop vs Mobile . . . 3

2 Užívateľské požiadavky **4**
2.1 Používateľské role . . . 4
2.2 Funkcionality jednotlivých rolí . . . 4
2.3 Cesta užívateľa . . . 5

3 Dátový model **6**
3.1 Popis entít . . . 7

4 Technologické požiadavky **10**
4.1 Client-side . . . 10
4.2 Server-side . . . 10
4.3 Interface Client-side a Server-side . . . 10
4.4 Hosting a deployment . . . 10

5 Časový plán **11**

6 Future work **13**

1 Základný popis aplikácie

1.1 Motivácia

V poslednej dobe narastá počet investicií, ktoré smerujú do nehnuteľností a nie vždy sú využívané širokou verejnosťou, ale iba úzkou skupinou ľudí, napríklad rodinami, priateľmi alebo komunitami. Najväčším problémom rezervačných systémov ako booking.com, airbnb.com, či vrbo.com je ich zameranie na širokú verejnosť s vysokými poplatkami, nedostatočnou kontrolou nad rezerváciami a obmedzenými možnosťami prispôsobenia pre majiteľov nehnuteľností. Navyše, tieto platformy často neberú do úvahy špecifické potreby a požiadavky majiteľov nehnuteľností, čo môže viesť k frustrácii a neefektívnemu spravovaniu rezervácií.

1.2 Ciele našej aplikácie

Cieľom našej aplikácie je vytvoriť privátny rezervačný systém, ktorý bude slúžiť pre konkrétnu skupinu užívateľov, napríklad pre rodinu, priateľov, malú komunitu alebo aj širšiu verejnosť. Hlavnou motiváciou je poskytnúť majiteľovi prehľad o obsadenosti apartmánov a nehnuteľností, umožniť jednoduché spravovanie rezervácií a odstrániť závislosť na externých platformách, ktoré môžu byť nákladné a neflexibilné.

1.3 Cieľová skupina

Našou cieľovou skupinou sú majitelia nehnuteľností, ktorí hľadajú efektívny a flexibilný spôsob spravovania rezervácií bez závislosti na externých platformách. Títo majitelia môžu byť jednotlivci, rodiny, priatelia alebo malá komunita, ktorá chce mať kontrolu nad svojimi rezerváciami a nechcú platiť vysoké poplatky externým platformám.

1.4 Desktop vs Mobile

Dizajn aplikácie bude responzívny tak, aby bola aplikácia prístupná a plnohodnotne použiteľná na desktopových zariadeniach, tabletoch aj mobilných telefónoch. Na desktopových zariadeniach bude hlavné menu zobrazené horizontálne, zatiaľ čo na mobilných zariadeniach bude riešené vo vertikálnej podobe. Obsah jednotlivých stránok bude usporiadaný pomocou grid layoutu, ktorý umožní flexibilné rozloženie prvkov podľa veľkosti obrazovky a zabezpečí dobrú použiteľnosť na všetkých podporovaných zariadeniach. Na menších obrazovkách budú jednotlivé sekcie a ovládacie prvky usporiadané pod sebou tak, aby používateľ vedel aplikáciu pohodlne ovládať aj dotykovo.

2 Užívateľské požiadavky

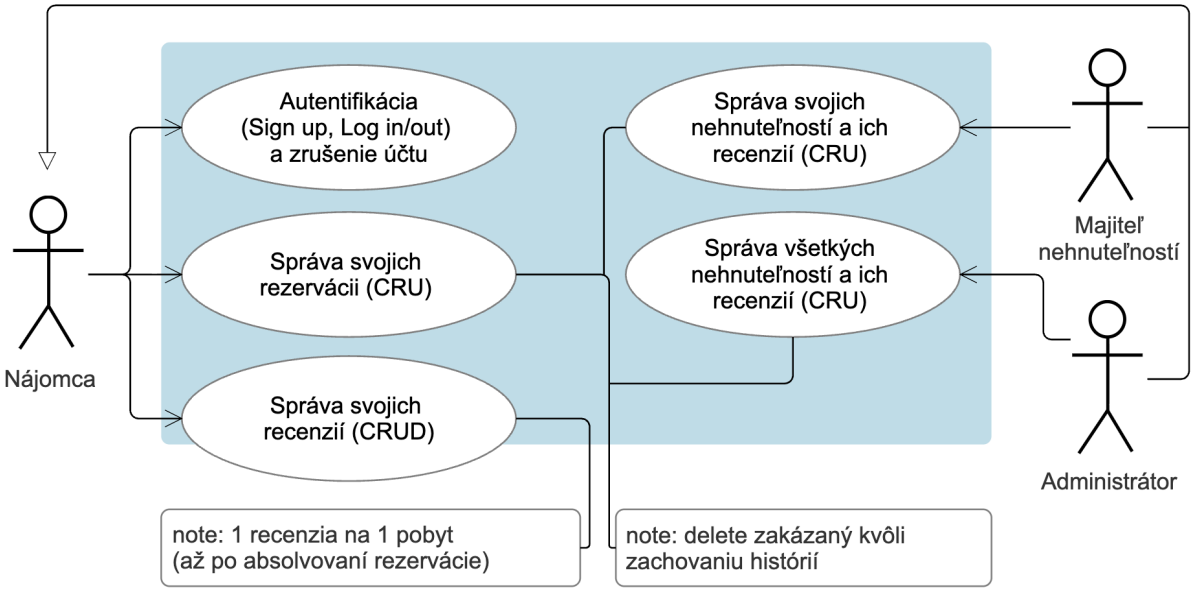
2.1 Používateľské role

Aplikácia bude rozlišovať následovné užívateľské role (zavedenie majiteľa nehnuteľnosti je motivovaný využívaním jednej inštancie aplikácie viacerými investormi):

- **Nájomca** - môže vyhľadávať a rezervovať nehnuteľnosti, spravovať svoje rezervácie a recenzie po uskutočnení prenájmu.
- **Majiteľ nehnuteľnosti** - má plný prístup k funkciám pre správu svojich nehnuteľností, rezervácií a užívateľov a má možnosť reagovať na ich recenzie.
- **Administrátor** - má plný prístup k všetkým funkciám aplikácie, vrátane správy užívateľov a nehnuteľností a všetkých existujúcich recenzií.

2.2 Funkcionality jednotlivých rolí

Diagram používa značenie **CRUD** (create, read, update, delete) pre zobrazenie základných operácií, ktoré môže každá rola vykonávať nad entitami v systéme. Pričom **CRU** znamená, že rola má právo vytvárať, čítať a aktualizovať danú entitu, ale nemá právo ju mazať.



Poznámky:

- autentifikácia nie je hlavnou funkcionalitou, ale je zahrnutá pre zabezpečenie prístupu k funkciám aplikácie a teda je prerekvizitou pre využívanie ostatných funkcionalít,
- update (aktualizácia) môže byť obmedzená na určité atribúty záznamu v relačnej tabulke,
- delete (mazanie) musí byť obmedzené pre rezervácie, kvôli sledovaniu histórie (zrušenie rezervácie = aktualizácia, nie delete).

2.3 Cesta užívateľa

Nájomca

Prihlási sa do aplikácie → prehliada si nehnuteľnosti → filtruje ich podľa svojich preferencií (lokácie, cena, vybavenie) → rezervuje vybranú nehnuteľnosť → spravuje svoje rezervácie (zobrazuje, aktualizuje, zruší) → vytvorí recenziu na nehnuteľnost po pobyte.

Majiteľ nehnuteľnosti

Dedí od **Nájomca** a navyše: Prihlási sa do aplikácie → pridá nehnuteľnosť → spravuje svoje nehnuteľnosti (zobrazuje, aktualizuje, maže) → spravuje rezervácie (zobrazuje, aktualizuje, zruší) → reaguje na recenzie.

Administrátor

Dedí od **Majiteľa nehnuteľnosti** a navyše: Prihlási sa do aplikácie → spravuje uživateľov, súbory, nehnuteľnosti, rezervácie a ich recenzie.

3 Dátový model

Z dôvodu rozšíriteľnosti aplikácie zavádzame **používateľské role** ako *abstraktné entity*, ktoré môžu byť definované pomocou systému oprávnení. Každá rola bude mať priradené určité oprávnenia, ktoré určujú, aké akcie môže používateľ s danou rolou vykonávať v aplikácii. Tento prístup nám umožní flexibilne pridávať nové role a oprávnenia bez nutnosti zásahu do základnej štruktúry systému.

Všeobecne bude *Nájomca*, *Majiteľ nehnuteľnosti* a *Administrátor* definovaný ako **User** s príslušnými oprávneniami.

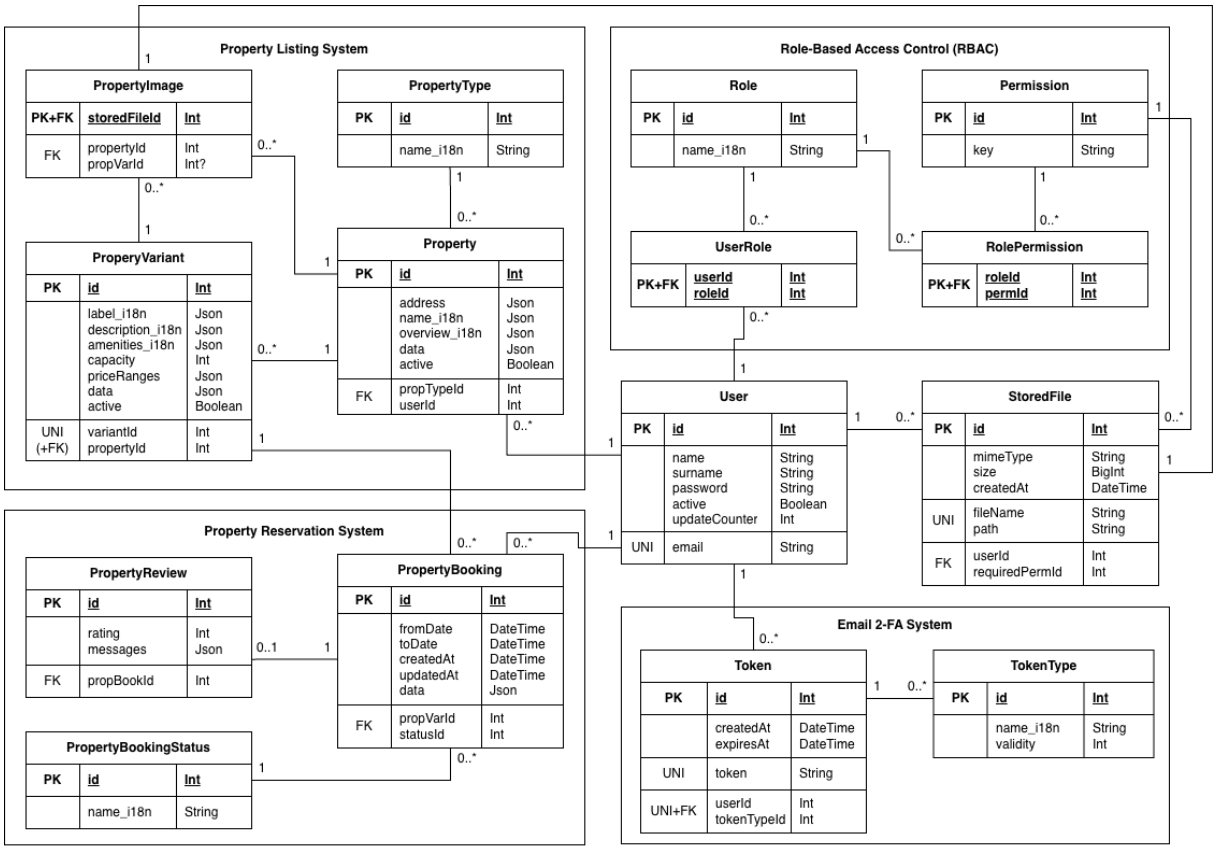
Dátový model taktiež definuje prácu so súbormi, ktoré môžu byť pripojené k rezerváciám alebo nehnuteľnostiam (zatiaľ iba ako obrázky).

Entitno-relačný diagram (ERD) znázorňujúci vzťahy medzi entitami v našom systéme je uvedený nižšie.

Poznámka: Anotácia typu napr. PK+FK znamená, že dané atributy sú PK aj FK.

Podobne, UNI+FK znamená, že dané atributy sú v kombinácii UNIQUE a každý z nich je FK, ale UNI (+FK) znamená, že dané atribúty sú v kombinácii UNIQUE, ale FK je iba atribút na úrovni 'newline' (+FK).

Teda riadok anotovaný UNI sa viaže na kombináciu atribútov, (.) na úroveň 'newline' a iné nevytvárajú kombinácie, ale hovoria o vlastnosti pre každý atribút zvlášť.*



3.1 Popis entít

User
Vyjadruje používateľa systému, ktorý môže byť *Nájomca*, *Majiteľ nehnuteľnosti* alebo *Administrátor*. Obsahuje štandardné atribúty ako *name, surname, password* a navyše:

- *email* - jednoznačný identifikátor používateľa (používaný pre prihlásenie),
- *active* - identifikuje, či je daný účet aktivný,
- *updateCounter* - identifikuje počet aktualizácii profilu, ktoré vynútia odhlásenie používateľa a opätovné prihlásenie (napr. zmena hesla) na ostatých zariadeniach.

StoredFile
Reprezentuje súbor uložený v systéme, ktorý môže byť pripojený k nehnuteľnostiam ako obrázok. Obsahuje nasledujúce atribúty:

- *mimeType, size, createdAt* - udržiavanie typu, veľkosti súbora a času vytvorenia,
- *filename, path* - jednoznačné identifikátory súbora (v kombinácii UNIQUE),
- *userId, requiredPermId* - foreign kľúče pre identifikáciu vlastníka súboru a potrebného oprávnenia pre prístup k súboru.

Token
Reprezentuje autentifikačný token, ktorý je používaný pre autentifikáciu a autorizáciu používateľov v systéme. Používa sa napríklad pri zmene hesla alebo aktivácii účtu, ktorý uživateľ dostane na svoju emailovú adresu s obmedzenou platnosťou. Obsahuje nasledujúce atribúty:

- *createdAt, expiresAt* - čas vytvorenia a expirácie tokenu,
- *token, tokenTypeId, userId* - konkrétna textová reprezentácia tokenu, typ tokenu a foreign kľúč pre identifikáciu vlastníka tokenu.

Navyše *token*, ale aj kombinácia *(tokenTypeId, userId)* musia byť UNIQUE, pretože každý používateľ môže mať len jeden aktívny token daného typu a zároveň by nemal existovať reťazec tokenu, ktorý je spoločný pre viacerých userov.

TokenType
Reprezentuje konkrétny typ tokenu, ktorý môže byť použitý, obsahuje atribúty ako:

- *name_i18n* - lokalizovaný názov typu tokenu,
- *validity* - doba platnosti tokenu v sekundách.

Role
Reprezentuje rolu, ktorá môže mať priradené určité oprávnenia s atribútom *name_i18n*, ktorý obsahuje lokalizovaný názov role.

UserRole
Definuje vzťah medzi používateľom a rolou, ktorá určuje, aké oprávnenia má používateľ v rámci danej role. Atribútami sú *userId, roleId*, ktoré sú primary kľúče a zároveň aj foreign kľúčami.

Permission
Definuje oprávnenie s atribútom *key*, ktorý identifikuje dané oprávnenie.

RolePermission
Definuje vzťah medzi rolou a oprávnením, ktorý určuje, aké oprávnenia má daná rola. Atribútami sú *roleId, permId*, ktoré sú primary kľúče a zároveň aj foreign kľúčami.

Property
Reprezentuje konkrétnu nehnuteľnosť, ktorá môže mať viac variant (ak nemá žiadnu, nie je možné ju zviditeľniť - active == false). Obsahuje nasledujúce atribúty:

- *address, name_i18n, overview_i18n* - adresa, lokalizovaný názov a popis nehnuteľnosti,
- *data* - voliteľné pole pre uloženie ďalších informácií o nehnuteľnosti,
- *active* - identifikuje, či je nehnuteľnosť aktívna (zobrazuje sa),
- *propTypeId, userId* - foreign kľúče pre identifikáciu typu nehnuteľnosti (byt, hotel, atd.) a vlastníka nehnuteľnosti.

PropertyType
Reprezentuje typ nehnuteľnosti, ktorý môže byť použitý pre kategorizáciu nehnuteľností a obsahuje atribút *name_i18n*, ktorý obsahuje lokalizovaný názov typu nehnuteľnosti.

PropertyVariant
Reprezentuje variant nehnuteľnosti (izba v hoteli, konkrétny apartmán alebo konkrétny priestor v rámci nehnuteľnosti). Obsahuje nasledujúce atribúty:

- *label_18n, description_18n, amenities_i18n* - lokalizovaný názov, popis variantu a vybavenia variantu,
- *capacity* - maximálny počet osôb, ktorý môže byť v tomto variante ubytovaných (iba pre "bytové jednotky"),
- *priceRanges* - pole pre uloženie ceny prenájmu vo formáte (od, do, cena/noc, [príplatky]),
- *data* - voliteľné pole pre uloženie ďalších informácií o variante,
- *active* - identifikuje, či je tento variant aktivný (zobrazuje sa),
- *variantId, propertyId* - jednoznačná kombinácia čísla variantu (1..*) a foreign kľúca pre identifikáciu nehnuteľnosti, ktorej tento variant patrí.

Cena rezervácie sa vypočíta na *priceRanges* súčtom cien za noc pre dané dátumové rozmedzie a (voliteľné) príplatkov. (Voliteľné) Príplatky môžu byť definované administrátorom: napr. cena na osobu / dieta, cena za upratovanie, parkovanie a podobne (budú vyžadovať rozšírenie dátového modelu).

PropertyImage
Reprezentuje obrázok pripojený ku konkrétnemu alebo všetkým variantom nehnuteľnosti. Obsahuje nasledujúce atribúty:

- *storedFileId* - primary a zároveň aj foreign kľúč pre identifiaciu súboru,
- *propertyId* - foreign kľúč pre identifikáciu nehnuteľnosti, ktorej tento obrázok patrí,
- *propVarId* - nullable foreign kľúč pre identifikáciu konkrétneho variantu, ktorému tento obrázok patrí (ak je null, patrí každému variantu nehnuteľnosti).

PropertyBooking
Reprezentuje konkrétnu rezerváciu variantu nehnuteľnosti konkrétnym používateľom. Po jej absoľvovaní umožní pridať k nehnuteľnosti recenziu. Obsahuje nasledujúce atribúty:

- *fromDate, toDate, createdAt, updatedAt* - dátum začiatku a konca rezervácie, dátum vytvorenia rezervácie a dátum aktualizácie rezervácie,
- *data* - voliteľné pole pre uloženie ďalších informácií o rezervacií,
- *propVarId, statusId* - foreign kľúče identifikujúce variant nehnuteľnosti a stav rezervácie.

PropertyBookingStatus
Reprezentuje stav rezervácie, ktorý môže byť použitý pre identifikáciu aktuálneho stavu (napr. "potvrdená", "zrušená", ...). Obsahuje atribút *name_i18n*, ktorý obsahuje lokalizovaný názov stavu rezervácie.

PropertyReview
Reprezentuje recenziu, ktorá môže byť priradená ku už absolvovanej rezervacií variantu nehnuteľnosti. Obsahuje následujúce atribúty:

- *rating* - hodnotenie variantu nehnuteľnosti v rozsahu 1..5,
- *messages* - json pole pre uloženie správ recenzie medzi nájomcom a majitelom nehnuteľnosti; formát *(userId: Integer, createdAt: DateTime, updatedAt: DateTime, message: String)*, pričom musí existovať a zároveň byť prvá správa od nájomcu.
- *propBookId* - foreign kľúč pre identifikáciu rezervácie, ku ktorej je recenzie priradená.

4 Technologické požiadavky

4.1 Client-side

- **Jazyk**: TypeScript, **Framework**: Vue 3, **Routing**: Vue Router *verzia ^4.5.1*
- **State management**: Pinia *verzia ^3.0.3*
- **HTTP komunikácia**: Axios *verzia ^1.10.0* (knižnica pre AJAX)
- **UI / štýly**: PrimeVue *verzia ^4.3.6*, Tailwind CSS *verzia ^4.1.13*
- **Internacionalizácia**: Vue I18n *verzia ^11.1.9*
- **Build tool**: Vite *verzia ^7.0.0*

Architektúrou bude **SPA (Single Page Application)** a navyše v priebehu implementácie bude testovaná na posledných verziách: *Chrome, Firefox, Safari a Edge.*

4.2 Server-side

- **Jazyk**: TypeScript, **Runtime**: Node.js 22, **Framework**: Express *verzia ^4.21.2*
- **ORM / databázová vrstva**: Prisma *verzia ^6.16.2*, **Databáza**: PostgreSQL *verzia 18.0*
- **Validácia**: Zod *verzia ^4.0.5*, Express Validator *verzia ^7.2.1*
- **Autentifikácia / bezpečnosť**: bcrypt *verzia ^6.0.0*, cookie-parser *verzia ^1.4.7*, CORS *verzia ^2.8.5*, express-rate-limit *verzia ^7.5.1*
- **Upload súborov**: Multer *verzia ^1.4.5-lts.2*
- **Emailová komunikácia**: Nodemailer *verzia ^7.0.5*
- **Konfigurácia prostredia**: dotenv *verzia ^17.1.0*
- **Testovanie**: Vitest *verzia ^4.0.18*, Supertest *verzia ^7.1.3*

4.3 Interface Client-side a Server-side

Ako už knižnica *Axios* napovedá, komunikácia bude prebiehať pomocou **REST API** a **JSON** formátu. Server bude poskytovať API endpointy pre všetky potrebné operácie (CRUD) nad entitami systému, ako aj autentifikáciu a autorizáciu. Klient bude tieto endpointy využívať na získavanie a manipuláciu s dátami, ktoré následne zobrazí používateľovi.

4.4 Hosting a deployment

Použitý bude súkromný server na adrese *https://codeanchor.eu*, ktorý bude mať continous integration a deployment pipeline nastavenú pomocou GitHub WebHooks. Na serveri beží Ubuntu 22.04 LTS s Kubernetesom pre orchestráciu kontajnerov, ktoré budú obsahovať client-side a server-side aplikáciu.

5 Časový plán

Plánujem pracovať iteratívne a to konkrétne tak, že celý projekt si rozdelím na menšie časti a každú z nich budem implementovať v určitom týždni semestra. Zároveň si jeden deň vyhradím na code review práce z predchádzajúceho týždňa, aby som sa uistil, že všetko funguje správne a že kód je čistý a dobre zdokumentovaný.

5. týždeň semestra

Implementácia základnej štruktúry stránky, navigácie, základných komponentov a lokalizácie (slovenčina, angličtina). Zároveň implementácia autentifikácie používateľa, teda registrácie, prihlasovania, zmeny hesla, zmeny údajov účtu a zrušenia účtu.
Odhad času: približne 16 hodín

- základný layout, navigácia, routing: 4 h
- základné reusable komponenty: 3 h
- lokalizácia: 2 h
- autentifikácia a formuláre: 5 h
- testovanie, code review a opravy: 2 h

6. týždeň semestra

Implementácia stránky pre zobrazenie nehnuteľností, recenzií a ich variantov vrátane dostupnosti a vytvorenia rezervácie.
Odhad času: približne 16 hodín

- zoznam nehnuteľností a variantov: 4 h
- detail variantu / nehnuteľnosti: 3 h
- zobrazenie recenzií a priemerného hodnotenia: 2 h
- vytvorenie rezervácie: 5 h
- testovanie, code review a opravy: 2 h

7. týždeň semestra

Implementácia stránky pre správu rezervácií, histórie rezervácií a možnosti ich zrušenia. Zároveň implementácia možnosti vytvorenia recenzie po absolvovanom pobyte.
Odhad času: približne 18 hodín

- aktuálne rezervácie, ich história a zrušenie: 8 h
- zoznam recenzií s filtrovaním, pridanie recenzie, správa recenzií: 8 h
- testovanie, code review a opravy: 2 h

8. týždeň semestra

Implementácia stránky pre správu nehnuteľností, teda pridanie novej nehnuteľnosti, úpravu existujúcej a zoznam vlastnených nehnuteľností.
Odhad času: približne 14 hodín

- zoznam vlastnených nehnuteľností: 3 h
- formulár na vytvorenie nehnuteľnosti: 4 h
- editácia nehnuteľnosti a variantov: 5 h
- testovanie, code review a opravy: 2 h

9. týždeň semestra

Implementácia stránky pre správu používateľov vrátane zobrazenia používateľov, ich rezervácií, vlastnených nehnuteľností, súborov a možnosti úpravy ich údajov.
Odhad času: približne 14 hodín

- zoznam a detail používateľov: 6 h
- rezervácie / nehnuteľnosti / súbory používateľa: 6 h
- testovanie, code review a opravy: 2 h

10 - 13. týždeň semestra

Finalizácia projektu, optimalizácia výkonu, dokončenie testovania a príprava finálnej verzie.
Odhad času: približne 28 hodín

- integračné testovanie: 10 h
- opravy chýb, code review a refaktoring: 16 h
- príprava finálnej verzie: 2 h

6 Future work

Dátový model nám umožňuje pridávať nové typy nehnuteľností, role, statusy rezervácií ako aj typy tokenov.

V budúcnosti by bolo zaujímavé rozšíriť systém o administráciu uvedených typov a vytvoriť tak možnosť pre administrátorov systému jednoducho, bez zmeny kódu obohatiť aplikáciu o nové funkcionality.

Predpokladám, že počas implementácie táto časť bude viac obohatená.